

ДИСЦИПЛИНА	Программирование (полное наименование дисциплины без сокращений)
ИНСТИТУТ	Институт тонких химических технологий им. М.В. Ломоносова
КАФЕДРА	Кафедра информационных систем в химической технологии полное наименование кафедры)
ВИД УЧЕБНОГО МАТЕРИАЛА	Практические занятия (в соответствии с пп.1-11)
ПРЕПОДАВАТЕЛЬ	Серебренников Н.П. (фамилия, имя, отчество)
СЕМЕСТР	2-й семестр, 1-й курс бакалавриата, 2024-2025 учебный год (указать семестр обучения, учебный год)

Практическая работа № 6. Функции и модули

Цель работы

Изучить и закрепить работу с функциями в Python, включая функции более высокого порядка, встроенные функции (map, filter, sort), лямбда-выражения, замыкания, типы переменных (локальные, глобальные и nonlocal), а также научиться создавать и использовать собственные модули.

Задания

1. Функции

- Напишите функцию *calculate_area*, которая принимает длину и ширину прямоугольника и возвращает его площадь. Если передано только одно число, считайте, что это квадрат.

Используйте возможность сделать один необязательный параметр:

```
def calculate_area(a, b_____):
    if b is None:
        return _____
    return _____
```

```
print(calculate_area(5))      # 25
print(calculate_area(3, 4))   # 12
```

- **Напишите функцию `is_prime(n)`, которая проверяет, является ли число `n` простым:**

Вам необходимо проверить:

1) `n` больше единицы?

2) делится ли `n` нацело на какие-то числа от 2 до `sqrt(n)` включительно?

```
def is_prime(n):
    if ____:
        return False
    for i in range(____):
        if ____ == 0:
            return False
    return True
```

```
print(is_prime(7))    # True
print(is_prime(10))   # False
```

- **Напишите функцию `is_palindrome`, которая проверяет, является ли строка палиндромом (читается одинаково слева направо и справа налево).**

```
def is_palindrome(s):
    _____ ...

print(is_palindrome("radar"))    # True
print(is_palindrome("Python"))  # False
```

2. Функции высшего порядка

- Напишите функцию `apply_operation`, которая принимает функцию и два числа, применяет функцию к числам и возвращает результат.

В качестве функции можете создать свою, простейшую функцию, которая отдает в качестве результата сложение или умножение двух чисел.

```
def apply_operation(func, x, y):  
    return func(x, y)
```

```
def multiply(____):  
    return _____
```

```
print(apply_operation(multiply, 3, 4)) # 12
```

```
print(apply_operation(lambda x, y: x + y, 5, 7)) # 12
```

- Напишите функцию `compose(f, g, x)`, которая принимает две функции `f` и `g`, а затем вычисляет `f(g(x))`.

В качестве функций можете создать свои, простейшие функции, которые отдадут в качестве результата $(x+1)$ и умножение $(x*2)$.

```
def compose(f, g, x):  
    return f(g(x))
```

```
def add_one(n):
```

```
def multiply_by_two(n):
```

```
print(compose(multiply_by_two, add_one, 3)) # (3 + 1)  
* 2 = 8
```

```
print(compose(add_one, multiply_by_two, 3)) # 7
```

3. Встроенные функции высшего порядка

- У вас есть список строк с названиями городов:

```
cities = ["London", "Paris", "Berlin", "Amsterdam",  
"Moscow", "Oslo", "Madrid"]
```

1. Напишите функцию **to_uppercase**, которая принимает строку и возвращает её в верхнем регистре. Можете использовать встроенную функцию **upper()** для строки.
2. Используйте **map()** и функцию **to_uppercase**, чтобы получить список названий городов в верхнем регистре.
3. Напишите функцию **has_o**, которая проверяет, содержит ли строка букву 'o' (в любом регистре). Используйте оператор **in** для проверки условия вхождения и функцию **lower()** для строки.
4. Используйте **filter()** и функцию **has_o**, чтобы оставить только те города, в названии которых есть буква 'o'.
5. Напишите функцию **length_sort**, которая возвращает длину строки. Можете для определения длины строки использовать стандартную функцию **len()**.
6. Используйте **sorted()** и функцию **length_sort**, чтобы отсортировать города по длине их названий.

```
cities = ["London", "Paris", "Berlin", "Amsterdam",  
"Moscow", "Oslo", "Madrid"]
```

```
# 1. Функция для перевода строки в верхний регистр
```

```
def to_uppercase(city):
```

```
...
```

```
# 2. Применяем map() с to_uppercase
```

```
uppercase_cities = list(map(_____))
```

```
print(uppercase_cities)
```

```
# ['LONDON', 'PARIS', 'BERLIN', 'AMSTERDAM', 'MOSCOW',  
'OSLO', 'MADRID']
```

```
# 3. Функция для проверки наличия буквы 'o' (регистр не  
важен)
```

```
def has_o(city):
```

```
...
```

```
# 4. Применяем filter() с has_o
```

```
cities_with_o = _____
```

```

print(cities_with_o)

# ['London', 'Moscow', 'Oslo', 'Madrid']

# 5. Функция для сортировки по длине строки
def length_sort(city):
    _____

# 6. Применяем sorted() с length_sort
sorted_cities = sorted(cities, key=_____)
print(sorted_cities)

# ['Oslo', 'Paris', 'Berlin', 'London', 'Madrid',
'Moscow', 'Amsterdam']

```

4. Лямбда-функции

- **Отсортируйте список кортежей (имя, возраст) по возрасту с помощью лямбда-функции:**

```

people = [("Alice", 25), ("Bob", 20), ("Charlie", 30)]
result = sorted(people, key=lambda x: _____)
print(result)

```

- **Используйте лямбда-функцию для возведения чисел из списка в квадрат:**

```

numbers = [1, 2, 3, 4]
result = list(map(lambda x: _____, numbers))
print(result)  # [1, 4, 9, 16]

```

- **Отсортируйте список словарей по нескольким ключам: сначала по возрасту (по возрастанию), затем по имени (по возрастанию):**

```

people = [
    {"name": "Alice", "age": 25},
    {"name": "Bob", "age": 20},
    {"name": "Charlie", "age": 25},
    {"name": "Diana", "age": 20}
]

```

Для сортировки сложной структуры (списка словарей) используйте сортировку ***sorted()*** по ключу из лямбда функции, в ней правильно укажите название полей сортировки, используйте функцию ***ord()*** (возвращает номер символа Unicode) для сортировки по имени.

```
sorted_people = sorted(  
    people,  
    key=_____ ) ,  
)  
print(sorted_people)
```

5. Замыкания функций

- Напишите функцию ***multiply_by(n)***, которая возвращает функцию, умножающую входное число на n:

```
def multiply_by(n):  
    def multiplier(x):  
        return _____  
    return _____
```

```
double = multiply_by(2)  
print(double(5))
```

- Напишите функцию ***password_generator(length)***, которая принимает длину пароля и возвращает вложенную функцию. Эта вложенная функция при каждом вызове будет генерировать случайный пароль заданной длины (объяснение функционала после шаблона кода):

```
import random  
import string
```

```
def password_generator(length):  
    def generate():  
        characters = _____ # Буквы и цифры
```

```

        return ''.join(random._____ for _ in
range(_____))

    return generate # Возвращаем вложенную функцию

# Создаем генератор паролей длиной 8 символов
gen8 = password_generator(8)

# Генерируем несколько паролей
print(gen8()) # Например, 'AlbC3dEf'
print(gen8()) # Например, 'Xy7ZkP2Q'

# Создаем генератор паролей длиной 12 символов
gen12 = _____
print(_____) # Например, '1aB2cD3eF4G5'

```

- `password_generator(length)` создает замыкание, где `length` сохраняется в памяти.
- `generate()` использует `length`, чтобы создать строку случайных символов.
- Возвращенная функция `generate` при каждом вызове генерирует новый пароль заданной длины.

6. Типы переменных (global, local, nonlocal)

- **Создайте функцию для управления банковским аккаунтом:**

Описание:

Глобальная переменная **bank_balance** — общий баланс банка.

Функция **create_account(initial_balance)** создает банковский счет (замыкание), используя локальную переменную **balance**.

Внутренняя функция `account(action, amount=0)` позволяет:

- "deposit" — пополнить счет (увеличивает `balance`).
- "withdraw" — снять деньги (уменьшает `balance`, если хватает средств).

- "total_bank_balance" — показать общий баланс банка (global bank_balance).

```
bank_balance = 100000 # Глобальный баланс банка

def create_account(initial_balance):
    balance = initial_balance # Локальная переменная
    (баланс счета)

    def account(action, amount=0):
        _____ balance # Используем _____ для
изменения локальной переменной
        _____ bank_balance # Используем _____
для доступа к общему балансу

        if action == "deposit":
            balance = _____
            bank_balance = _____
            return f"Депозит: +{amount}, Баланс счета:
{balance}"

        elif action == "withdraw":
            if amount <= _____:
                balance = _____
                bank_balance = _____
                return f"Снятие: -{amount}, Баланс
счета: {balance}"
            else:
                return "Ошибка: недостаточно средств!"

        elif action == "total_bank_balance":
            return f"Общий баланс банка:
{bank_balance}"
```



```

        else:
            return "Неизвестная операция!"

    return account # Возвращаем вложенную функцию
(замыкание)

# Создаем два счета
alice_account = create_account(5000)
bob_account = create_account(3000)

# Операции с Alice
print(alice_account(____)) # Депозит: +2000,
Баланс счета: 7000

print(alice_account(____)) # Снятие: -3000,
Баланс счета: 4000

# Операции с Bob
print(bob_account(____)) # Снятие: -1000,
Баланс счета: 2000

# Общий баланс банка
print(alice_account(____)) # Покажет
обновленный баланс банка

```

7. Создание своего модуля и его использование

Создайте модуль mymath.py, содержащий функции add(a, b) и multiply(a, b). Импортируйте и используйте их в другом файле.

```

# mymath.py

def add(a, b):
    return _____

```

```
def multiply(a, b):  
    return _____
```

Из другого файла вызовите свой модуль. **НО перед этим допишите в path переменную окружения путь до папки вашего модуля.**

Чтобы это сделать, можете разово использовать код:

```
import sys  
  
s = 'c:\\<путь до папки с модулем>\\'  
  
sys.path.append(s)  
  
print (sys.path)
```

Затем импортируйте свой модуль и примените функции из него:

```
import mymath  
  
print(mymath.add(3, 7))    # 10  
print(mymath.multiply(4, 5)) # 20
```

Вопросы для самоконтроля

- Как создать функцию, которая принимает разное количество аргументов?
- Чем отличается return от print внутри функции?
- Как сделать аргумент функции опциональным?
- Можно ли вернуть из функции несколько значений? Если да, то как?
- Что такое функция высшего порядка? Приведи пример.
- Как передать функцию как аргумент в другую функцию?
- В чем разница между map и filter?
- В чем отличие лямбда-функции от обычной?
- Когда стоит использовать лямбда-функции, а когда — нет?
- Что делает ключевое слово nonlocal и когда его нужно использовать?
- Чем отличается локальная переменная от глобальной?
- Как получить доступ к глобальной переменной внутри функции?
- Как импортировать конкретную функцию из модуля?